Marketing Video Generator MVP – 1 Hour Development Specification

1 Project Overview

Build a web application with four connected nodes that transforms uploaded product images and basic prompts into marketing videos posted to Instagram. The system must be functional within one hour, prioritizing core workflow over polish.

2 Architecture: Sequential Four-Node Pipeline

2.1 Node 1: Image Upload Component

Create a React component that handles exactly two image uploads. Use HTML5 file input with drag-and-drop functionality. Store uploaded files in browser memory using the `FileReader` API to convert images to base64 data URLs. Display preview thumbnails immediately after upload. Implement basic validation for file types (JPG, PNG) and size limits (under 5 MB each). Pass the base64 image data to the next node through React state management.

Rapid Development Note For rapid development, skip cloud storage entirely. Keep images in browser memory throughout the session. This approach eliminates external dependencies and API setup time while maintaining functionality for the MVP demonstration.

2.2 Node 2: Prompt Input Component

Build a simple `textarea` component that captures user creative intent. Implement character counting with a 200-character limit to keep prompts focused. Include placeholder text like "Describe the person and scene you want for your marketing video." Store the prompt text in React state and pass it to the enhancement system along with the uploaded images.

Validation Add basic validation to ensure the prompt contains at least 10 characters before allowing progression to the next node. This prevents empty or inadequate prompts from reaching the enhancement system.

2.3 Node 3: Prompt Enhancement Engine

This component transforms basic user input into detailed Veo3-compatible JSON. Create a JavaScript function that analyzes the user prompt and generates a complete specification following Veo3's expected format.

MVP Logic Build the enhancement logic using string parsing and keyword detection. For the MVP, implement basic rule-based mapping rather than complex AI analysis. For example:

- If the prompt contains words like *professional* or *business*, set the mood to formal and lighting to bright commercial style.
- If the prompt mentions *casual* or *lifestyle*, adjust the aesthetic accordingly.

Template & Substitution Create a base JSON template with placeholder values, then implement substitution functions that fill in specific sections based on prompt analysis. The enhancement system should always generate valid JSON that includes platform targeting for Instagram Reels, appropriate cinematography settings for product showcase, and action sequences that highlight both uploaded products within the 6–8 second timeframe.

Core Enhancement Logic (Example)

```
// Core enhancement function (MVP-friendly)
function enhancePrompt(userPrompt, imageData1, imageData2) {
  const baseTemplate = {
    platform_target: "instagram_reel_9x16",
    shot: {
      type: "medium shot with product focus",
      framing: "subject and products clearly visible in 9:16 aspect ratio",
      composition_notes: "both products prominently featured"
    },
    subject: {
      identity: extractSubjectFromPrompt(userPrompt),
      appearance: determineAppearanceStyle(userPrompt),
      wardrobe: "clean, modern styling that complements products",
      props: "user uploaded products integrated naturally"
    },
    setting: {
      location: inferSettingFromPrompt(userPrompt),
      time_of_day: "optimal lighting for product showcase",
      weather: "clear conditions",
      set_dressing: "minimal, clean background"
    },
    color_and_light: {
      palette: "bright, commercial lighting optimized for product visibility",
      key_light: "even, professional lighting",
      fill_rim: "soft fill to eliminate harsh shadows",
      contrast_grain: "clean commercial aesthetic"
    },
    action: {
      blocking: "natural product demonstration and interaction",
      performance: "confident, engaging product presentation",
      beats: [
        { t: "0-2s", event: "establish scene with both products visible" },
        { t: "2-5s", event: "primary product interaction and showcase" },
```

```
      { t: "5-8s", event: "secondary product feature with final composition" }
    ],
    loopable: true
  },
  camera: {
    movement: "smooth movement highlighting products",
    lens_look: "sharp focus on products with appealing depth",
    stabilization: "professional steady movement",
    focus: "products remain clearly visible throughout"
  },
  style: {
    genre: "commercial product showcase",
    mood: determineMoodFromPrompt(userPrompt),
    texture: "polished, professional marketing aesthetic",
    avoid_hyperreal: true
  },
  audio: {
    ambience: ["subtle commercial background"],
    foley: ["natural product interaction sounds"],
    music: "upbeat commercial background music",
    dialogue: generateOptionalDialogue(userPrompt),
    mix_notes: "clear commercial audio mix"
  },
  cinematic_controls: {
    pacing: "dynamic but clear product visibility",
    transitions: "smooth cuts maintaining product focus",
    thumbnail_hint: "select frame with optimal product visibility"
  },
  negatives: [
    "cluttered backgrounds that distract from products",
    "poor product visibility or lighting",
    "fast movements that blur product details",
    "AI artifacts or distortions"
  ],
  notes: {
    duration_hint: "6-8 seconds optimized for Instagram",
    one_shot_only: true,
    priority: "product clarity and commercial appeal"
  }
};

  return populateTemplate(baseTemplate, userPrompt, imageData1, imageData2);
}

// --- Helper function stubs (MVP keyword-based) ---
function extractSubjectFromPrompt(p) {
  // e.g., "professional man", "casual woman", fallback:
  const m = p.match(/\b(man|woman|model|person)\b/i);
  return m ? `a ${m[0]}` : "a person";
```

```
}
function determineAppearanceStyle(p) {
  if (/\b(luxury|elegant|premium|formal|professional)\b/i.test(p)) return "formal,
      premium styling";
  if (/\b(casual|lifestyle|street|relaxed)\b/i.test(p)) return "casual lifestyle styling"
      ;
  if (/\b(athletic|sport|fitness)\b/i.test(p)) return "athletic, active styling";
  return "clean, modern styling";
}
function inferSettingFromPrompt(p) {
  if (/\b(outdoor|street|park|beach)\b/i.test(p)) return "outdoor lifestyle setting";
  if (/\b(studio|indoor|office)\b/i.test(p)) return "clean modern studio";
  return "minimal, modern environment";
}
function determineMoodFromPrompt(p) {
  if (/\b(professional|business|formal|premium|luxury)\b/i.test(p)) return "professional
      and aspirational";
  if (/\b(casual|lifestyle|relaxed|cozy)\b/i.test(p)) return "casual, friendly lifestyle"
      ;
  if (/\b(athletic|sport|energetic|dynamic)\b/i.test(p)) return "energetic and dynamic";
  return "polished commercial";
}
function generateOptionalDialogue(p) {
  // Keep short; many Reels are music-only
  return "";
}
function populateTemplate(tpl, userPrompt, img1, img2) {
  // Attach references if needed; for MVP, we just echo presence
  tpl.subject.props = "two user-uploaded products are showcased prominently";
  tpl.notes.prompt_summary = userPrompt.slice(0, 200);
  tpl.assets = { image1_present: !!img1, image2_present: !!img2 };
  return tpl;
}
```

2.4 Node 4: Veo3 Integration Component

Create a component that takes the enhanced JSON specification and uploaded images, then calls the Veo3 API to generate the marketing video. Since you have Veo3 access, implement the API call using `fetch` with proper error handling.

User Feedback Display a loading spinner during video generation with status messages like "Analyzing your products" and "Generating your marketing video." Handle the Veo3 response and display the generated video once complete. Provide basic error handling with user-friendly messages if generation fails.

Retries For the MVP, implement a simple retry mechanism that attempts the API call up to three times before displaying an error message. Store the generated video URL in React state for

the Instagram posting component.

2.5 Node 5: Instagram Posting Interface (Simulated)

Build a component that displays the generated video and provides Instagram posting functionality. For the MVP, implement a simplified version that shows the video preview and includes a "Post to Instagram" button.

Simulation Rationale Since Instagram Graph API integration requires OAuth setup and Business account verification, implement this as a simulated posting interface for the MVP demonstration. Display a success message and mock posting confirmation rather than actual Instagram integration. This allows you to demonstrate the complete workflow without spending time on complex authentication setup.

Placeholders Include placeholder elements for where real Instagram integration would connect, such as account selection and caption input fields. This demonstrates the intended functionality while keeping development time under one hour.